CI/CD for Machine Learning using MLRun

Team 06

Aditya Prakash Borate
23110065@iitgn.ac.in
23110065

Swayam Giridhar Borate
23110066@iitgn.ac.in
23110066

GITHUB REPO

1. Dataset Preparation:

- About Breast Cancer Dataset:
  - Features: 30 features describing various characteristics of cell nuclei present in breast cancer biopsies.
  - Target: Binary classification, where 0 represents benign and 1 represents malignant.
- Python Code for Data Loading:

```
from sklearn.datasets import load_breast_cancer
data = load_breast_cancer()
df = pd.DataFrame(data.data, columns=data.feature_names)
df["target"] = data.target
```

2. MLRun Project and Component Functions

- MLRun Project Initialization (main.ipynb or Colab link)

A new MLRun project was initialized using the `mlrun.new_project()` method.

```
project = mlrun.new_project("lab9-breast-cancer", "./",
user_project=True, overwrite=True, init_git=False)
```

This project serves as the workspace for defining and managing all pipeline components.

- Data Preparation Script (data_prep.py)

  The data preparation logic was encapsulated in a separate Python script. The script loads the breast cancer dataset, processes the features and target labels, and prepares the data for model training.

```python
import mlrun
from sklearn.datasets import load_breast_cancer
import pandas as pd

@mlrun.handler(outputs=["dataset", "label_column"])
def data_loader(context, format="csv"):
    """
    Loads the Breast Cancer dataset, prepares a DataFrame, and logs it as an MLRun Dataset artifact.
    """
    context.logger.info("Loading Breast Cancer dataset using sklearn")
    data = load_breast_cancer()
    df = pd.DataFrame(data.data, columns=data.feature_names)
    df["target"] = data.target

    context.logger.info(f"Dataset shape: {df.shape}")
    context.logger.info(f'Saving dataset artifact using key "breast_cancer_dataset" to {context.artifact_path}')

    # Log the DataFrame as a Dataset artifact
    context.log_dataset("breast_cancer_dataset", df=df, format=format, index=False)

    context.logger.info("Dataset artifact logged successfully")

    return df, "target"

if __name__ == "__main__":
    with mlrun.get_or_create_ctx("data-prep-local", upload_artifacts=True) as context:
        data_loader(context)
```

- Model Training Script (trainer.py)

  Model training was handled in the trainer.py script. The data was split into training and testing subsets using a 80/20 split. A Random Forest Classifier was selected as the model, and the training process was instrumented using MLRun's

apply_mlrun function to enable automatic logging and artifact tracking.

```
import mlrun
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from mlrun.frameworks.sklearn import apply_mlrun

# Define the main training function
def train_model(
    context: mlrun.run.MLClientCtx,
    dataset: mlrun.DataItem,
    label_column: str = "target",
    n_estimators: int = 100,
    max_depth: int = 5,
    random_state: int = 42,
    model_name: str = "breast_cancer_classifier"
):
    """
    Trains a RandomForestClassifier model on the input dataset.

    Uses apply_mlrun to automatically log model, metrics, and
artifacts.
    """
    context.logger.info("Starting model training")

    # Load DataFrame from the input DataItem
    context.logger.info(f"Loading data from {dataset.url}")
    df = dataset.as_df()
    context.logger.info(f"Data loaded, shape: {df.shape}")

    # Prepare features (X) and target (y)
    X = df.drop(label_column, axis=1)
    y = df[label_column]
    context.logger.info(f"Features shape: {X.shape}, Target
shape: {y.shape}")

    # Split data into training and testing sets
    X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.2, random_state=random_state)
    context.logger.info(f"Train set size: {X_train.shape[0]},
Test set size: {X_test.shape[0]}")

    # Initialize the RandomForestClassifier model
    model = RandomForestClassifier(
        n_estimators=n_estimators,
        max_depth=max_depth,
        random_state=random_state
    )
```

```
    context.logger.info(f"Initialized RandomForestClassifier
with n_estimators={n_estimators}, max_depth={max_depth}")

    # Log the model and metrics using apply_mlrun
    apply_mlrun(model=model, model_name=model_name,
x_test=X_test, y_test=y_test)
    model.fit(X_train, y_train)
    context.logger.info("Training completed and model artifacts
logged via apply_mlrun.")
```

- Model Serving Script (serving.py)

  Model serving was implemented by defining a class that extends `mlrun.serving.V2ModelServer`. The class includes the `load()` and `predict()` methods, allowing seamless model deployment and prediction serving.

```
import mlrun
from cloudpickle import load
import numpy as np
from typing import List

class BreastCancerModel(mlrun.serving.V2ModelServer):
    """
    MLRun Serving Class for the Breast Cancer RandomForest
model.
    Inherits from V2ModelServer for standard model serving
capabilities.
    """

    def load(self):
        """
        Loads the trained RandomForest model from the specified
path using get_model().
        """
        self.context.logger.info(f"Loading model from
{self.model_path}...")
        model_file, extra_data = self.get_model('.pkl')
        self.model = load(open(model_file, 'rb'))
        self.context.logger.info("Model loaded successfully.")


    def predict(self, body: dict) -> List:
        """
        Receives prediction requests and returns model
predictions.
        Assumes that the 'inputs' key in the body contains a
list of feature lists.
        """
```

```
        feats = np.asarray(body['inputs'])
        self.context.logger.info(f"Received {feats.shape[0]}
samples for prediction.")
        results: np.ndarray = self.model.predict(feats)
        return results.tolist()
```

- Workflow Definition (workflow.py)

  The end-to-end ML pipeline was constructed using the `@dsl.pipeline decorator`. This workflow orchestrates three stages: data ingestion, model training with hyperparameter tuning, and model deployment.

  Hyperparameter tuning was conducted over `n_estimators: [10, 100, 200]` and `max_depth: [2, 5, 10]`, selecting the model with the highest accuracy.

  Model deployment was achieved using `mlrun.deploy_function`, targeting the base Kubernetes image mlrun/mlrun.

```
import mlrun
from kfp import dsl

@dsl.pipeline(name="breast-cancer-demo")
def pipeline(model_name="breast-cancer-classifier"):

    ingest = mlrun.run_function(
        "load-breast-cancer-data",
        name="load-breast-cancer-data",
        params={"format": "pq", "model_name": model_name},
        outputs=["dataset"],
    )

    train = mlrun.run_function(
        "trainer",
        inputs={"dataset": ingest.outputs["dataset"]},
        hyperparams={
            "n_estimators": [10, 100, 200],
            "max_depth": [2, 5, 10]
        },
        selector="max.accuracy",
        outputs=["model"],
    )

    deploy = mlrun.deploy_function(
        "serving",
        models=[{"key": model_name, "model_path":
train.outputs["model"], "class_name": "BreastCancerModel"}],
```

```
        mock=True
    )
```

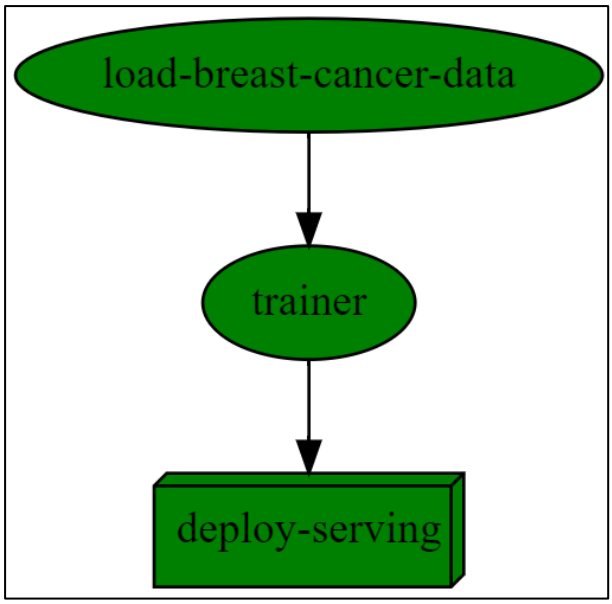
3. Workflow Execution and Artifacts

- Workflow Graph

  Upon execution using

```
ru_id = project.run(
    workflow_path="./workflow.py",
    arguments={"model_name": "breast-cancer-classifier"},
    watch=True, local=no_k8s)
```

  , a visual representation of the pipeline was generated, illustrating the connections between data ingestion, model training, and deployment stages.



- Data Preparation Artifact

  The dataset was loaded using the `load-breast-cancer-data` function, which outputs the dataset as an artifact in MLRun. The artifact was retrieved from the `gen_data_run object` and

displayed using gen_data_run.artifact("dataset").as_df().head().

```
gen_data_run = project.run_function("load-breast-cancer-data", params={"format":"csv"}, local=True)
```

36.8s Python

```
> 2025-04-14 15:08:42,342 [warning] it is recommended to use k8s secret (specify secret_name), specifying the aws_access_key/aws_secret_key
> 2025-04-14 15:08:42,427 [info] Storing function: {"db":"http://localhost:30070","name":"load-breast-cancer-data","uid":"caaed6bb93a946f68a
> 2025-04-14 15:08:43,050 [info] Handler was not provided running main (data_prep.py)
Running: ['c:\\Users\\adibo\\anaconda3\\envs\\sttai\\python.exe', '-u', 'data_prep.py']
> 2025-04-14 15:09:04,735 [info] logging run results to: http://localhost:30070
> 2025-04-14 15:09:04,871 [info] Loading Breast Cancer dataset using sklearn
> 2025-04-14 15:09:04,913 [info] Dataset shape: (569, 31)
> 2025-04-14 15:09:04,914 [info] Saving dataset artifact using key "breast_cancer_dataset" to s3://mlrun/projects/lab9-breast-cancer-adibo/a
> 2025-04-14 15:09:13,881 [info] Dataset artifact logged successfully
```

project	uid	iter	start	state	kind	name	labels	inputs	parameters	results	artifacts
lab9-breast-cancer-adibo	...f3b08f	0	Apr 14 09:39:04	completed	run	load-breast-cancer-data	kind=local owner=adibo host=Adi		format=csv	label_column=target	breast_cancer_dataset dataset

```
gen_data_run.artifact("dataset").as_df().head()
```

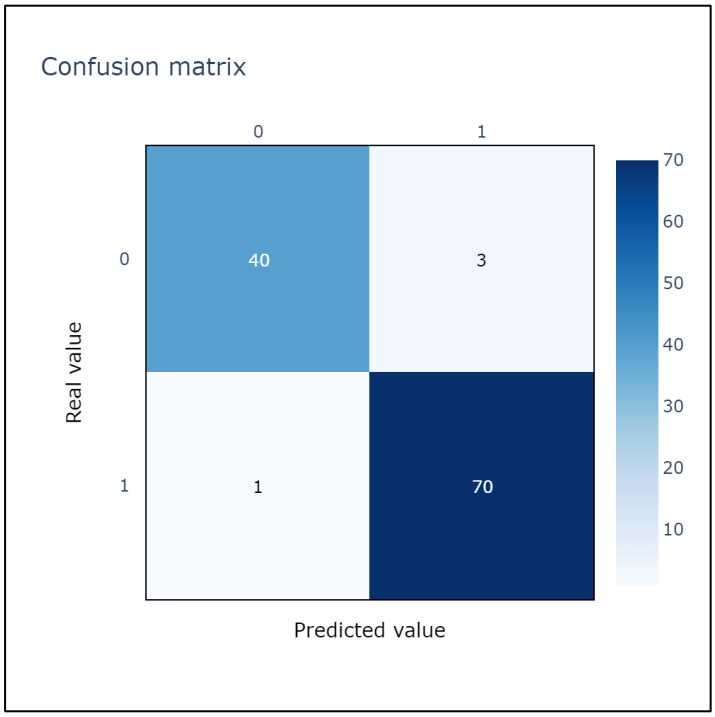
2.8s Python

	mean radius	mean texture	mean perimeter	mean area	mean smoothness	mean compactness	mean concavity	mean concave points	mean symmetry	mean fractal dimension	...	worst texture	worst perimeter	worst area	w smooth
0	17.99	10.38	122.80	1001.0	0.11840	0.27760	0.3001	0.14710	0.2419	0.07871	...	17.33	184.60	2019.0	0.
1	20.57	17.77	132.90	1326.0	0.08474	0.07864	0.0869	0.07017	0.1812	0.05667	...	23.41	158.80	1956.0	0.
2	19.69	21.25	130.00	1203.0	0.10960	0.15990	0.1974	0.12790	0.2069	0.05999	...	25.53	152.50	1709.0	0.
3	11.42	20.38	77.58	386.1	0.14250	0.28390	0.2414	0.10520	0.2597	0.09744	...	26.50	98.87	567.7	0.
4	20.29	14.34	135.10	1297.0	0.10030	0.13280	0.1980	0.10430	0.1809	0.05883	...	16.67	152.20	1575.0	0.

5 rows × 31 columns

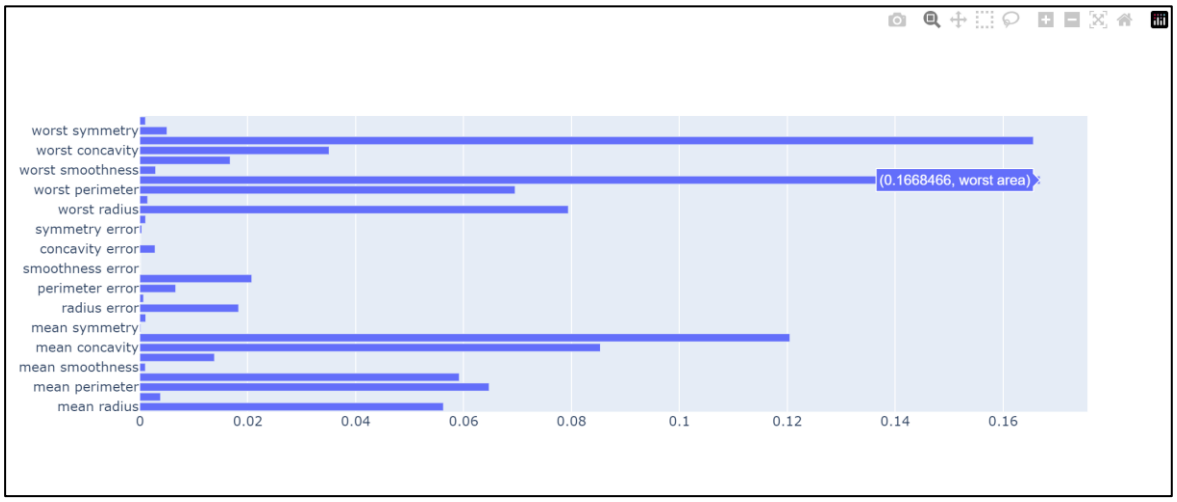
- Training Metrics - Confusion Matrix

During model training, MLRun automatically generated performance artifacts, including the confusion matrix. This is the confusion matrix of the best model found after hyper-parameter tuning.



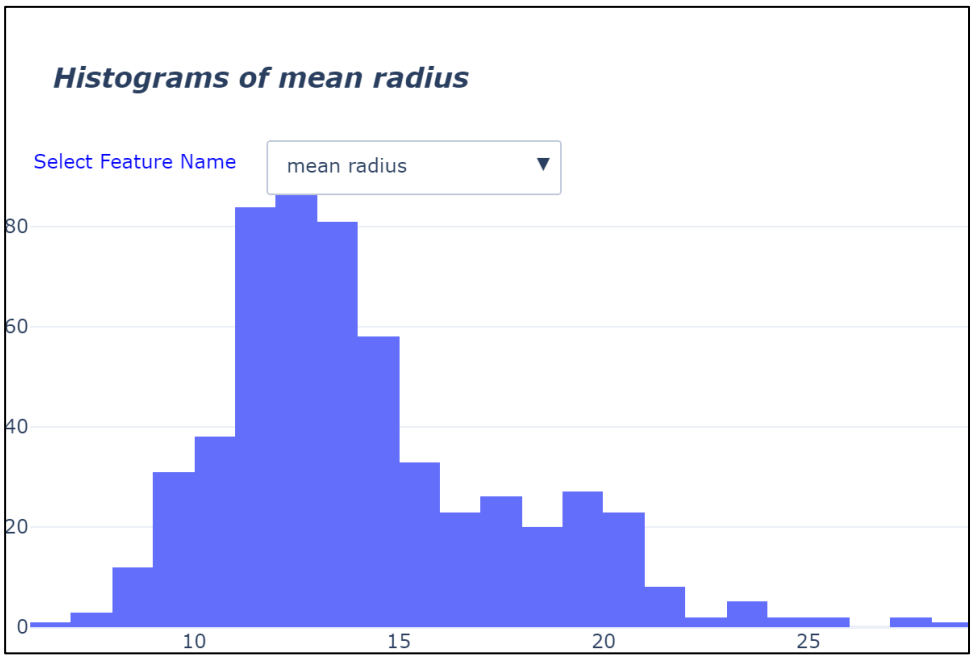
- Feature Selection Artifact

  MLRun logged feature importances via Random Forest's built-in mechanism. This artifact suggests that 'worst area' is the feature of highest importance.

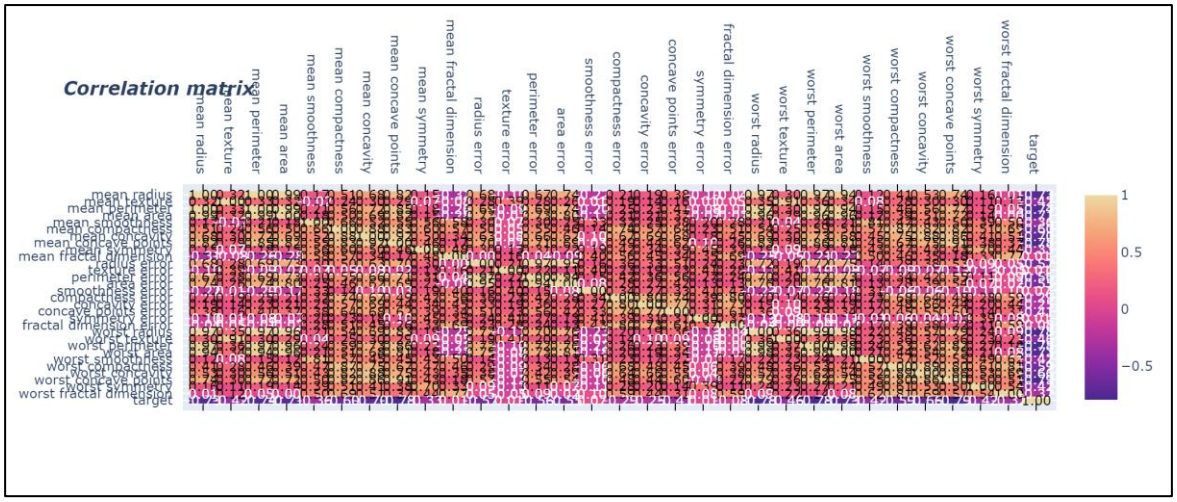


#Extras

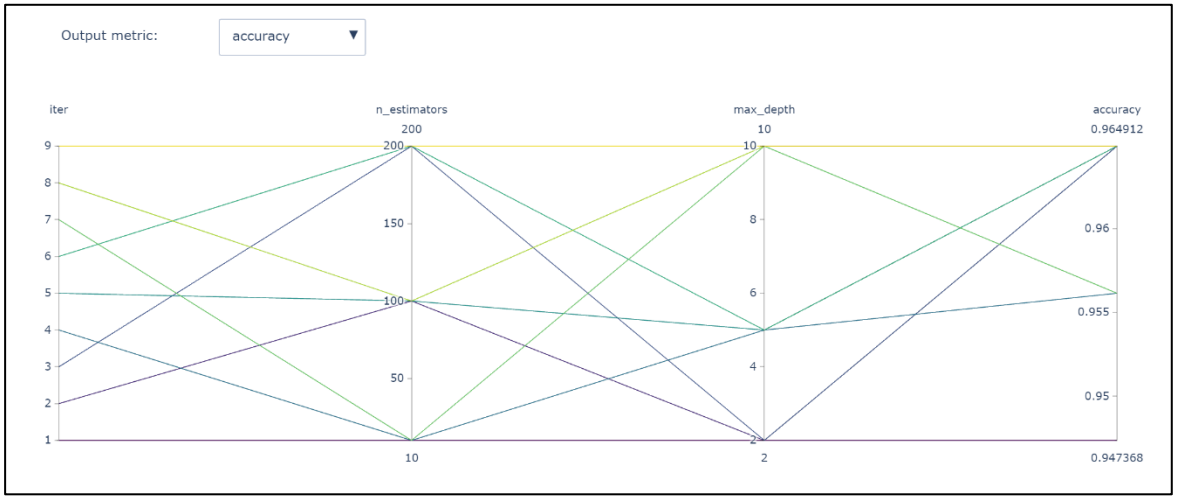
- Histogrmas of features of the dataset



- Correlation matrix of the dataset



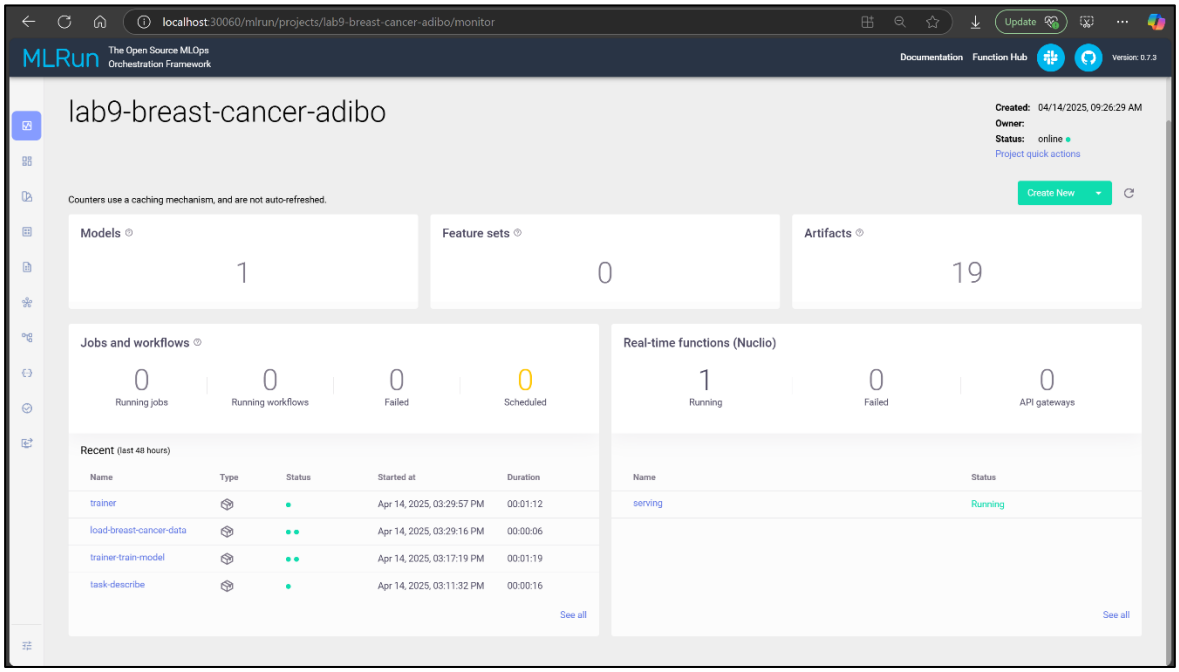
- Parallel Coordinates Artifact

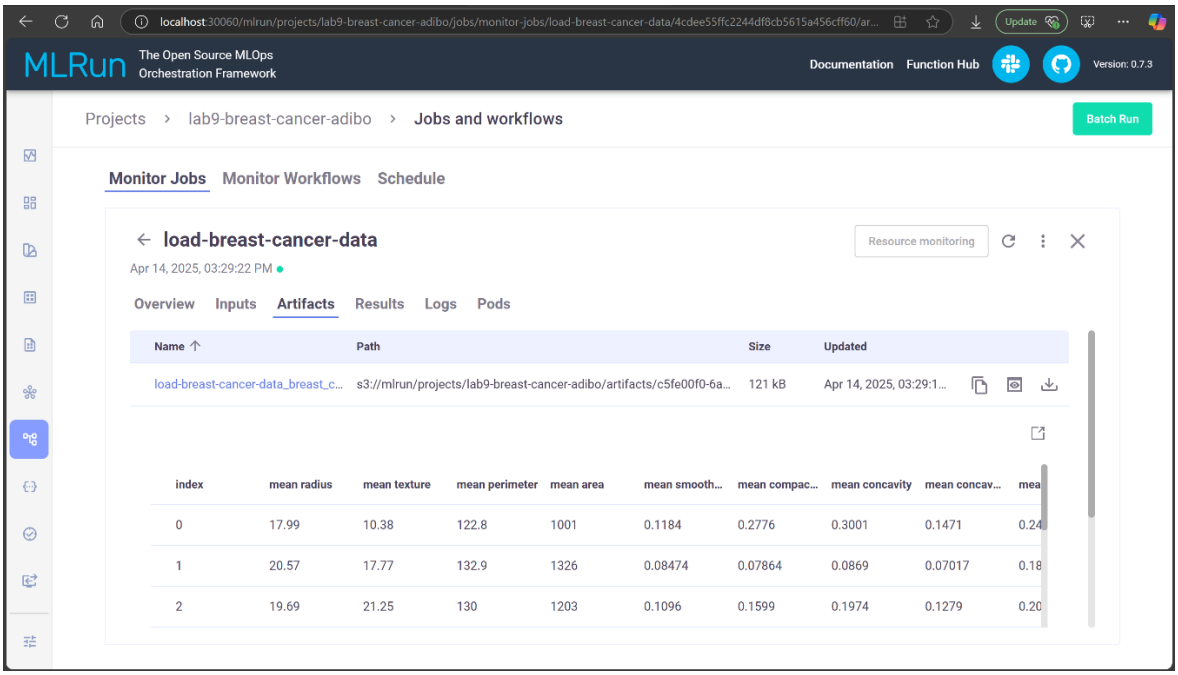


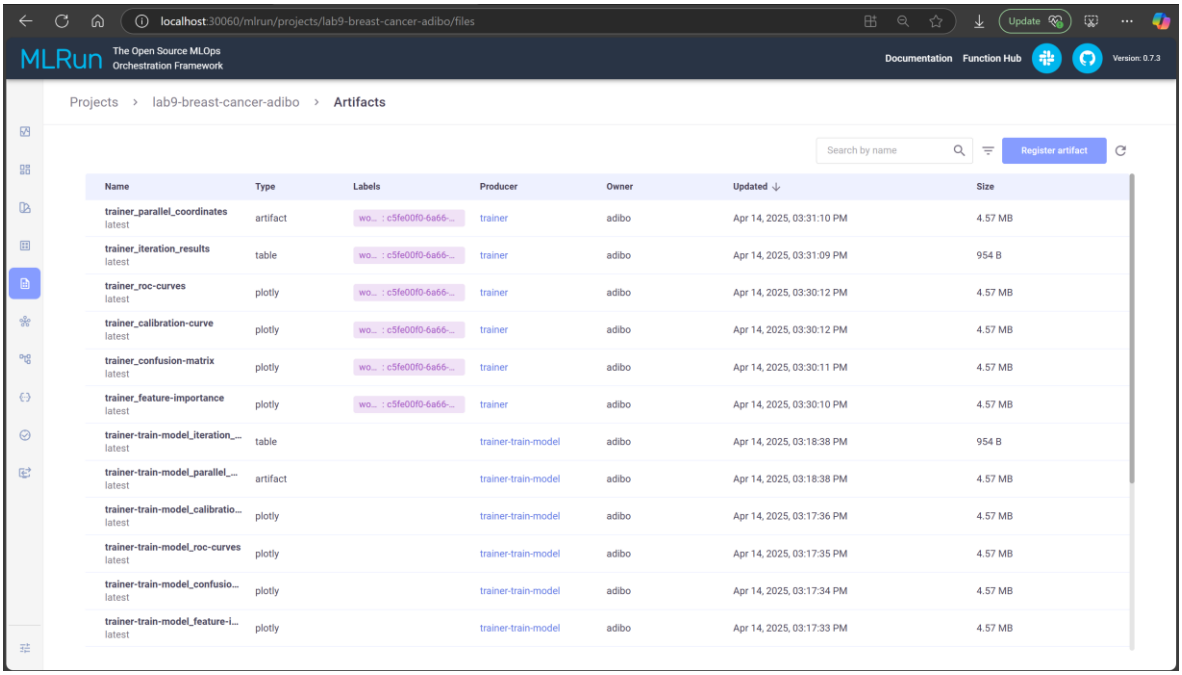
This suggests keeping accuracy as the metric, the best set of hyperparameters are:
Iteration 2: n_estimators: 100, max_depth = 2
Iteration 3: n_estimators: 200, max_depth = 2
Iteration 5: n_estimators: 100, max_depth = 5
Iteration 6: n_estimators: 200, max_depth = 5
Iteration 8: n_estimators: 100, max_depth = 10
Iteration 9: n_estimators: 200, max_depth = 10

All performing equally well with an accuracy of 0.9649122807017544 on test dataset.

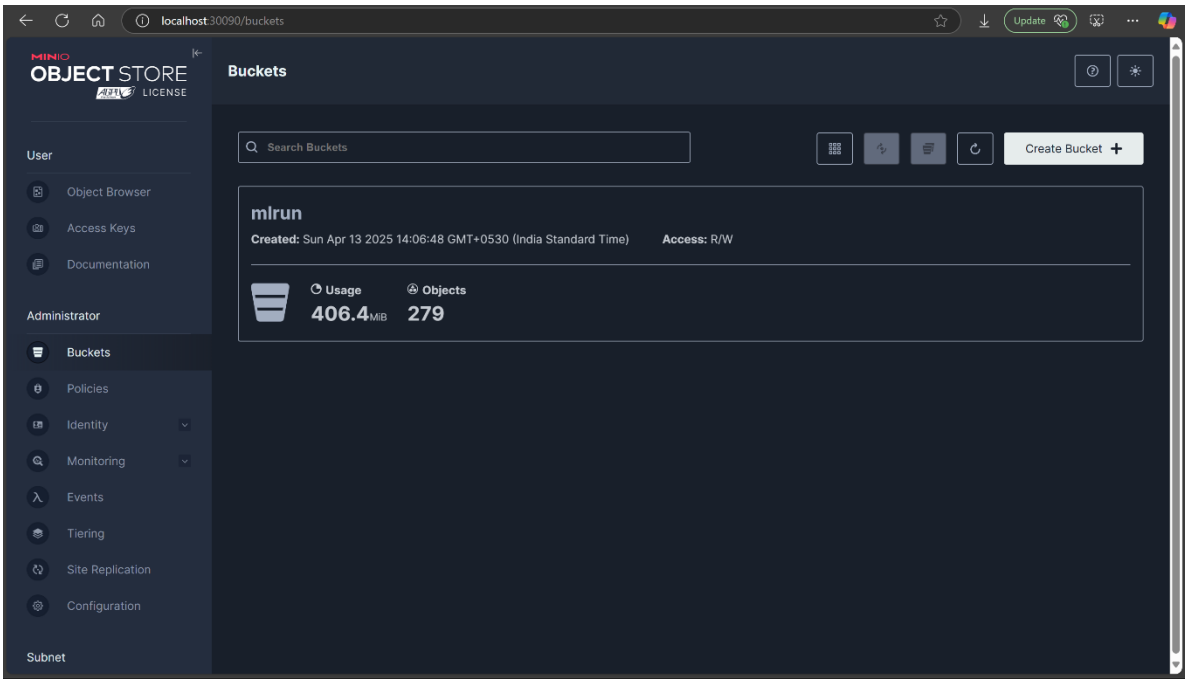
- MLRun UI:


localhost:30060/mlrun/projects/lab9-breast-cancer-adibo/monitor
MLRun
The Open Source MLOps Orchestration Framework
Documentation
Function Hub
lab9-breast-cancer-adibo
Created: 04/14/2025, 09:26:29 AM
Owner:
Status: online
Project quick actions
Counters use a caching mechanism, and are not auto-refreshed.
Create New
Models
1
Feature sets
0
Artifacts
19
Jobs and workflows
0 Running jobs
0 Running workflows
0 Failed
0 Scheduled
Real-time functions (Nuclio)
1 Running
0 Failed
0 API gateways
Recent (last 48 hours)
Name Type Status Started at Duration
trainer Apr 14, 2025, 03:29:57 PM 00:01:12
load-breast-cancer-data Apr 14, 2025, 03:29:16 PM 00:00:06
trainer-train-model Apr 14, 2025, 03:17:19 PM 00:01:19
task-describe Apr 14, 2025, 03:11:32 PM 00:00:16
See all
Name Status
serving Running
See all


localhost:30060/mlrun/projects/lab9-breast-cancer-adibo/jobs/monitor-jobs/load-breast-cancer-data/4cdee55ffc2244df8cb5615a456cff60/ar...
MLRun
The Open Source MLOps Orchestration Framework
Documentation
Function Hub
Projects › lab9-breast-cancer-adibo › Jobs and workflows
Batch Run
Monitor Jobs
Monitor Workflows
Schedule
load-breast-cancer-data
Resource monitoring
Apr 14, 2025, 03:29:22 PM
Overview
Inputs
Artifacts
Results
Logs
Pods
Name
Path
Size
Updated
load-breast-cancer-data_breast_c...
s3://mlrun/projects/lab9-breast-cancer-adibo/artifacts/c5fe00f0-6a...
121 kB
Apr 14, 2025, 03:29:1...
index mean radius mean texture mean perimeter mean area mean smooth... mean compac... mean concavity mean concav... mea
0 17.99 10.38 122.8 1001 0.1184 0.2776 0.3001 0.1471 0.24
1 20.57 17.77 132.9 1326 0.08474 0.07864 0.0869 0.07017 0.18
2 19.69 21.25 130 1203 0.1096 0.1599 0.1974 0.1279 0.20


localhost:30060/mlrun/projects/lab9-breast-cancer-adibo/files
MLRun
The Open Source MLOps Orchestration Framework
Documentation
Function Hub
Projects › lab9-breast-cancer-adibo › Artifacts
Search by name
Register artifact
Name
Type
Labels
Producer
Owner
Updated
Size
trainer_parallel_coordinates latest artifact trainer adibo Apr 14, 2025, 03:31:10 PM 4.57 MB
trainer_iteration_results latest table trainer adibo Apr 14, 2025, 03:31:09 PM 954 B
trainer_roc-curves latest plotly trainer adibo Apr 14, 2025, 03:30:12 PM 4.57 MB
trainer_calibration-curve latest plotly trainer adibo Apr 14, 2025, 03:30:12 PM 4.57 MB
trainer_confusion-matrix latest plotly trainer adibo Apr 14, 2025, 03:30:11 PM 4.57 MB
trainer_feature-importance latest plotly trainer adibo Apr 14, 2025, 03:30:10 PM 4.57 MB
trainer-train-model_iteration_... latest table trainer-train-model adibo Apr 14, 2025, 03:18:38 PM 954 B
trainer-train-model_parallel_... latest artifact trainer-train-model adibo Apr 14, 2025, 03:18:38 PM 4.57 MB
trainer-train-model_calibratio... latest plotly trainer-train-model adibo Apr 14, 2025, 03:17:36 PM 4.57 MB
trainer-train-model_roc-curves latest plotly trainer-train-model adibo Apr 14, 2025, 03:17:35 PM 4.57 MB
trainer-train-model_confusio... latest plotly trainer-train-model adibo Apr 14, 2025, 03:17:34 PM 4.57 MB
trainer-train-model_feature-i... latest plotly trainer-train-model adibo Apr 14, 2025, 03:17:33 PM 4.57 MB

- Minio UI


localhost:30090/buckets
MINIO
OBJECT STORE
LICENSE
Buckets
User
Object Browser
Access Keys
Documentation
Administrator
Buckets
Policies
Identity
Monitoring
Events
Tiering
Site Replication
Configuration
Subnet
Search Buckets
Create Bucket +
mlrun
Created: Sun Apr 13 2025 14:06:48 GMT+0530 (India Standard Time)
Access: R/W
Usage
406.4 MiB
Objects
279

***